

Week 2 short benchmark report: analytical query optimization

Method was being monitored

The full-data run was made on 2026-09-19 and is identified as 20260919T143454Z-9d921a51. Thirteen paired experiments were executed: seven comparisons for the four techniques required by the assignment and six comparisons between canonical queries and product-backed forms. The pinned source was integration version 0 from batch 563b32da, containing 9,554,576 trips. It is the same source on both sides of each experiment, which means the optimized side does not obtain a smaller or more convenient dataset unless changing the source to a declared materialized product is itself the experiment.

One factor is changed in every pair: a partition predicate, an explicit cache, a broadcast hint, the AQE setting, or the source product. Each variant is warmed once and measured three times through `spark.sql()` followed by `collect()`. Non-cache experiments alternate baseline and optimized ordering. Cache comparisons are different, because once Spark finds a matching cached logical plan, it can use that cache inside a later query labelled as baseline. For these two cases, all baseline samples are taken before cache construction, then construction is timed by `count()`, then the optimized samples are taken. The ordering is less symmetrical but the uncached side is genuinely uncached.

Results are compared before speed is stated. Counts, keys and strings require exact equality, while floating-point measures use relative tolerance 1e-9 and absolute tolerance 1e-6. The broadcast pair is also checked against canonical Q1 because both tested forms are base-table rewrites. All thirteen comparisons passed. All comparisons passed, and no mismatch was permitted to acquire a speedup.

The partition range is the half-open New York interval `[2024-02-01, 2024-03-01)`. Baselines apply the local-time expression; optimized variants add equivalent physical `pickup_date` limits. Broadcast tests set `spark.sql.autoBroadcastJoinThreshold=-1` and disable AQE so the baseline remains a real sort-merge join. Plans are saved using `EXPLAIN FORMATTED` together with the final executed plan.

Environment, environment used.

The machine ran Windows build 26200 on an Intel i9-13900HX with approximately 32 GB RAM. Software was Python 3.11.9, Java 21.0.7, Spark 4.2.0 and Delta 4.4.0. Spark used `local[4]`, a 4 GiB JVM heap, 128 shuffle partitions, UTC session timezone and ANSI mode. AQE and a 10 MiB automatic broadcast threshold were defaults except where the selected experiment overrode them.

Median results

The table reports median wall-clock seconds from three measured executions. Speedup is baseline divided by optimized time.

Experiment	Technique	Baseline (s)	Optimized (s)	Speedup
q1_partition_pruning	partition pruning	1.842	1.469	1.25x
q6_partition_pruning	partition pruning	0.813	0.634	1.28x
q4_cache_projection	cache	4.871	4.860	1.00x
q6_cache_projection	cache	1.439	1.034	1.39x
q1_broadcast_zones	broadcast join	8.051	5.563	1.45x
q4_aqe	AQE	28.151	4.911	5.73x
q6_aqe	AQE	1.451	1.281	1.13x
q1_product_taxi_zone_statistics	data product	4.026	0.434	9.27x
q2_product_weather_impact_summary	data product	0.656	0.329	1.99x
q3_product_air_quality_impact_summary	data product	1.539	1.179	1.31x
q4_product_weather_impact_summary	data product	4.378	0.841	5.21x
q5_product_daily_mobility_summary	data product	0.943	0.316	2.99x
q6_product_daily_mobility_summary	data product	1.060	0.367	2.89x

Four products together use 172,329 bytes and four data files. The integrated table uses 1,058,134,566 bytes and 91 files, so current product storage overhead is 0.016%. Full refresh lasts 142.5 seconds: 48.7, 33.5, 35.1 and 25.3 seconds for the four products. Running one complete six-query round from products saves 9.14 seconds compared with canonical SQL, giving an approximate refresh break-even of 16 full rounds. The arithmetic is an operational guide rather than a guarantee of future timings.

Cache construction is also outside reuse timing. Q4's five-column projection costs 4.59 seconds and 77.1 MB across 11 partitions; Q6's one-column projection costs 0.76 seconds and 19.5 MB. Both caches were released after their experiments.

Execution-plan findings and plan findings

The partition-pruned variants have three `PartitionFilters` entries, whereas their baselines have none. The added filter skips unrelated date partitions while the original local-date condition still protects meaning. Q1's base-table baseline shows `SortMergeJoin`; the hinted version shows `BroadcastHashJoin`, removing shuffle of the large Taxi side. Cached variants alone contain `InMemoryTableScan`. AQE-enabled results show `isFinalPlan=true` and `AQEShuffleRead`; Q4 contains 15 adaptive shuffle-read nodes. Raw Exchange counts across AQE on and off are not compared because an adaptive plan prints embedded sub-plans and those counts are not equivalent objects.

These facts were extracted automatically and the full plans were retained. Therefore the report does not assume an optimization happened because a configuration was requested. The plan has to show it, or it has to be visible there.

Performance discussion

The largest improvement overall is materialized Q1 at 9.27x. Among the four required techniques, AQE on Q4 is clearly largest at 5.73x. Q4 crosses 259 zones with 2,183 weather hours and aggregates the resulting values three times. When AQE is disabled, 128 shuffle partitions are written even after the data becomes small, and fixed partition work controls runtime. Runtime statistics let AQE combine those partitions.

Caching Q4 provides 1.00x, effectively no effect, despite the separate 4.59-second build and 77.1 MB footprint. Although the query reads a filtered trip projection twice, those scans are not its limiting operation; cross join and aggregation dominate. Q6 cache reuse is better at 1.39x but still has a build cost. This is why caching was not made a platform default.

Partition pruning produces moderate 1.25x and 1.28x gains. The February filter removes 62 of 91 date partitions, yet the queries read only one or two columns and Parquet already applies column pruning, so scan cost was smaller than might be expected. The broadcast hint improves the controlled join by 1.45x. In normal settings Spark can already broadcast a 265-row table automatically, and thus the experiment demonstrates why broadcast helps but does not argue for disabling the automatic threshold.

Q4 remains expensive in canonical form: 4.91 seconds even with AQE, compared with 0.84 seconds from `weather_impact_summary`. Q1 rebuilt from base tables remains 5.56 seconds after broadcasting because more than nine million records are labelled before they are reduced to 773 month-zone groups. Q3 has the smallest product gain, 1.31x, since its two-level median over the standardized air-quality table remains necessary after trip demand was already materialized.

The common cause is the reduction ratio. About 9.55 million trips collapse to 2,183 hours, 265 zones, three local months and six weather groups. Materialization and adaptive coalescing act directly on this shape, while cache and scan-level changes remove smaller portions of the work. This is an explanation grounded in these measurements, although it is not a universal rule for every Spark platform.

Ten-city outlook and limitations

For ten cities, analytical products need a city dimension and incremental refresh. A city-first and date-second partitioning arrangement would become reasonable once each city's partitions are sufficiently large. Around 2,650 zone rows remain safely broadcastable. AQE should remain helpful as post-aggregation sizes become uneven, while caching should still be enabled only after repeated scans are shown to matter. A full refresh scaling linearly from 142.5 seconds would approach half an hour, which makes incremental product maintenance more important. These are recommendations inferred from the current data shape; ten cities were not measured.

Only three samples were collected per variant. Consequently, 1.00x and 1.13x should be read as no clear improvement rather than highly precise ratios. OS caching is uncontrolled, and a single warm-up did not fully stabilize two baselines: broadcast Q1 declined from 10.13 to 8.05 to 7.27 seconds, while non-AQE Q4 rose from 20.01 to 28.15 to 28.45 seconds. Everything ran on one local machine, where shuffle does not traverse a network. Cluster behaviour could be different. Quite different, possibly, but it was not tested.

Reproduction and evidence location

Run `python -m scripts.run_query_benchmark` inside the project environment. `--list` prints experiment names and repeated `--experiment` options select a subset. Each run writes `data/benchmark/w2/<run_id>/results.json`, exact SQL under `sql/`, and formatted plus executed plans under `plans/`. `docs/w2_benchmark_timings.csv` contains all 78 measured timing rows from the reported run. Experiment definitions are in `src/dic_pipeline/query_benchmark.py`, while product-backed SQL is located in `src/dic_pipeline/sql/products/`. The retained evidence permits a reader to inspect the figures rather than having to accept only the summary, a summary which is also provided above.